In [3]:
```
# ================================
# Cell 1 — Imports, config, helpers
# ================================
from __future__ import annotations
import os, sys, re, gc, json, math, shutil, platform, random, logging
from pathlib import Path
from typing import List, Iterable, Optional
import numpy as np
import pandas as pd

# tqdm (optional)
try:
    from tqdm.auto import tqdm
    TQDM = True
except Exception:
    TQDM = False

# Logging
logging.basicConfig(level=logging.INFO, format="[%(levelname)s] %(message)s")
log = logging.getLogger("phase04")

# Determinism
SEED = int(os.getenv("SEED", "42"))
random.seed(SEED); np.random.seed(SEED)

# Outputs
OUTDIR = Path("outputs/phase04"); OUTDIR.mkdir(parents=True, exist_ok=True)
CACHE_DIR = Path("outputs/phase0/data_cache"); CACHE_DIR.mkdir(parents=True, exist_ok=True)

log.info(f"Python {sys.version.split()[0]} on {platform.system()} — SEED={SEED}")
log.info(f"Outputs -> {OUTDIR.resolve()}")
```

```
[INFO] Python 3.12.1 on Windows — SEED=42
[INFO] Outputs -> C:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\outputs\phase04
[INFO] Outputs -> C:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\outputs\phase04
```

In [10]:
```
import os
import shutil
from pathlib import Path
```

```python
# --- Make sure CACHE_DIR and list_matches are defined from a previous cell ---
CACHE_DIR = Path("outputs/phase0/data_cache")
# def list_matches(...): # This should be defined in a previous cell

# --- Start of FIX ---
# Initialize the ROLES_ONTO variable by trying to get it from an environment variable.
# If it's not set, this will correctly assign `None` to it.
ROLES_ONTO = os.getenv("ROLES_ONTO")
# --- End of FIX ---

# --- replace your mirror_adopt() with this silent-pin helper ---
def pin_local_copy(src: Path, target_name: str) -> str:
    """Copy src into a stable project cache path and return its POSIX path.
       Prints info (not warnings) to avoid noisy logs on Windows/OneDrive."""
    target = CACHE_DIR / target_name
    try:
        if str(src.resolve()) != str(target.resolve()):
            target.parent.mkdir(parents=True, exist_ok=True)
            shutil.copyfile(src, target)
        print(f"[info] Ontology pinned -> {target}")
        return target.resolve().as_posix()
    except Exception as e:
        print(f"[info] Using ontology in-place (pin copy failed): {e}")
        return src.resolve().as_posix()

# --- replace your ontology-missing block with this ---
# Now this `if` statement will work because ROLES_ONTO is guaranteed to exist.
if ROLES_ONTO is None or not Path(ROLES_ONTO).exists():
    # Assuming list_matches is defined in a previous cell
    nearby = list_matches(["*IT*Job*Roles*Skills*.csv","*roles*skills*.csv","*IT*Roles*.csv"])
    if not nearby:
        raise FileNotFoundError("Roles ontology CSV not found (e.g., /mnt/data/IT_Job_Roles_Skills.csv).")
    # prefer /mnt/data then shortest path
    nearby.sort(key=lambda p:(0 if str(p).lower().startswith(str(Path('/mnt/data')).lower()) else 1, len(str(p))))
    chosen = nearby[0]
    ROLES_ONTO = pin_local_copy(chosen, "IT_Job_Roles_Skills.csv")
else:
    # if it exists, also pin quietly for stability
    ROLES_ONTO = pin_local_copy(Path(ROLES_ONTO), "IT_Job_Roles_Skills.csv")
```

```
[info] Ontology pinned -> outputs\phase0\data_cache\IT_Job_Roles_Skills.csv
```

In [11]:
```
# ---- Jobs resolver (so it doesn't show "(optional/not found)") ----
def build_minimal_jobs_canon(raw_jobs_path: Path) -> str:
    """Create a Phase-3-style jobs skills file for later phases."""
    print(f"[info] Building minimal jobs canonical file from: {raw_jobs_path}")
    df = pd.read_csv(raw_jobs_path, encoding="utf-8-sig", sep=None, engine="python")
    # heuristics: find columns
    title_cols = [c for c in df.columns if re.search(r"(title|role|position)", c, re.I)]
    desc_cols  = [c for c in df.columns if re.search(r"(desc|description|summary)", c, re.I)]
    skill_cols = [c for c in df.columns if re.search(r"skill", c, re.I)]
    def _maybe_json_list(x):
        if x is None or (isinstance(x,float) and math.isnan(x)): return []
        if isinstance(x, list): return [str(i).strip() for i in x if str(i).strip()]
        s = str(x).strip()
        if not s: return []
        try:
            v = json.loads(s);
            if isinstance(v, list): return [str(i).strip() for i in v if str(i).strip()]
        except Exception: pass
        return [p.strip(" '\"") for p in re.split(r"[;,]", s) if p.strip()]
    def first_nonempty(row, cols):
        for c in cols:
            v = row.get(c)
            if isinstance(v, str) and v.strip(): return v.strip()
        return ""
    out = pd.DataFrame({
        "job_title_any": df.apply(lambda r: first_nonempty(r, title_cols), axis=1) if title_cols else "",
        "job_desc_any":  df.apply(lambda r: first_nonempty(r, desc_cols),  axis=1) if desc_cols  else "",
        "skills_canon":  df.apply(lambda r: sum([_maybe_json_list(r.get(c)) for c in skill_cols], []), axis=1) if skill_cols e
    })
    out.insert(0, "job_id", np.arange(1, len(out)+1))
    PH3 = Path("outputs/phase3"); PH3.mkdir(parents=True, exist_ok=True)
    path = PH3 / "jobs_skills_normalised.csv"
    out.to_csv(path, index=False, encoding="utf-8-sig")
    print(f"[info] Jobs canonical -> {path}")
    return path.resolve().as_posix()

# Try to resolve from Phase-2/3 first
if not JOBS_CANON:
    for p in [
```

```
        Path("outputs/phase3/jobs_skills_normalised.csv"),
        Path("outputs/phase2/jobs_skill_counts_canonical.csv"),
        Path("/mnt/data/jobs_skills_normalised.csv"),
    ]:
        if p.exists():
            JOBS_CANON = p.resolve().as_posix()
            break

# If still missing, build from raw postings (mounted by you)
if not JOBS_CANON:
    raw_jobs = None
    # common location in your project
    for p in [Path("/mnt/data/all_job_post.csv"), Path("data/all_job_post.csv")]:
        if p.exists(): raw_jobs = p; break
    if raw_jobs:
        JOBS_CANON = build_minimal_jobs_canon(raw_jobs)
    else:
        # last resort: search
        cand = list_matches(["*job*post*.csv","*job*ads*.csv","*job*description*.csv"])
        cand = [p for p in cand if "all_job_post" in str(p).lower()] or cand
        if cand:
            JOBS_CANON = build_minimal_jobs_canon(cand[0])

# After all attempts: set a friendly message if still None
if not JOBS_CANON:
    print("[info] Jobs canonical still not found; continuing without it (not required in Phase 4).")
```

```
[info] Building minimal jobs canonical file from: c:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\data\raw\all_job_post.csv
[info] Jobs canonical -> outputs\phase3\jobs_skills_normalised.csv
```

In [12]:

```
# Load jobs if available
jobs_df = None
if 'JOBS_CANON' in globals() and JOBS_CANON:
    try:
        jobs_df = pd.read_csv(JOBS_CANON, encoding="utf-8-sig", sep=None, engine="python")
        # minimal columns for consistency
        if "job_id" not in jobs_df.columns: jobs_df["job_id"] = np.arange(1, len(jobs_df)+1)
        if "skills_canon" not in jobs_df.columns:
            # try to build it from any skill-like columns
            skill_cols = [c for c in jobs_df.columns if re.search(r"skill", c, re.I)]
            if skill_cols:
```

```
                def _maybe_json_list(x):
                    if x is None or (isinstance(x,float) and math.isnan(x)): return []
                    if isinstance(x, list): return [str(i).strip() for i in x if str(i).strip()]
                    s = str(x).strip()
                    if not s: return []
                    try:
                        v = json.loads(s);
                        if isinstance(v, list): return [str(i).strip() for i in v if str(i).strip()]
                    except Exception: pass
                    return [p.strip(" '\"") for p in re.split(r"[;,]", s) if p.strip()]
                jobs_df["skills_canon"] = jobs_df.apply(lambda r: sum([_maybe_json_list(r.get(c)) for c in skill_cols], []), a
        print(f"[info] Jobs loaded: {len(jobs_df)} rows from {JOBS_CANON}")
    except Exception as e:
        print(f"[info] Skipping jobs load (non-critical): {e}")
```

```
[info] Jobs loaded: 1167 rows from C:/Users/naikt/OneDrive/Desktop/tanmay_dissertation/outputs/phase3/jobs_skills_normalised.cs
v
```

In [13]:

```
# ============================
# Cell 2 — Robust path resolver
# ============================
import os, re, json, math, shutil
from pathlib import Path
import pandas as pd
import numpy as np

CACHE_DIR = Path("outputs/phase0/data_cache"); CACHE_DIR.mkdir(parents=True, exist_ok=True)

SEARCH_ROOTS = [
    Path.cwd(), Path.cwd()/ "data", Path.cwd()/ "outputs",
    Path.cwd().parent, Path.cwd().parent / "data",
    Path("/mnt/data"),
    Path.home() / "Desktop",
    Path.home() / "OneDrive" / "Desktop",
    Path.home() / "OneDrive" / "Documents",
    Path.home() / "OneDrive",
]

def list_matches(patterns, roots=SEARCH_ROOTS, limit=4000):
    hits, seen = [], set()
    for root in roots:
```

```
        try:
            for pat in patterns:
                for p in root.rglob(pat):
                    s = str(p.resolve())
                    if s not in seen:
                        seen.add(s); hits.append(p)
                        if len(hits) >= limit: return hits
        except Exception:
            continue
    return hits

def mirror_adopt(p: Path, target_name: str) -> str:
    target = CACHE_DIR / target_name
    try:
        shutil.copyfile(p, target)
        print(f"[warn] [mirror] Copied to {target}")
        return str(target.resolve())
    except Exception as e:
        print(f"[warn] [mirror-skip] Using original path (copy failed: {e})")
        return str(p.resolve())

def looks_like_resume_csv(csv_path: Path) -> bool:
    try:
        # read only header, tolerate BOM and odd separators
        df = pd.read_csv(csv_path, nrows=5, encoding="utf-8-sig", sep=None, engine="python")
    except Exception:
        return False
    cols = [c.lower() for c in df.columns]
    return any("skill" in c for c in cols) or any(re.search(r"(title|designation|role)", c) for c in cols)

# 1) Try Phase-3 outputs / ENV
RESUMES_CANON = os.getenv("RESUMES_CANON")
JOBS_CANON    = os.getenv("JOBS_CANON")
ROLES_ONTO    = os.getenv("ROLES_ONTO")

if not RESUMES_CANON:
    for p in [
        Path("outputs/phase3/resumes_skills_normalised.csv"),
        Path("outputs/phase3/resumes_with_skills_canonical.csv"),
        Path("outputs/phase7/resumes_with_skills_canonical.csv"),
        Path("/mnt/data/resumes_skills_normalised.csv"),
```

```
    ]:
        if p.exists(): RESUMES_CANON = str(p); break

if not JOBS_CANON:
    for p in [
        Path("outputs/phase3/jobs_skills_normalised.csv"),
        Path("outputs/phase2/jobs_skill_counts_canonical.csv"),
        Path("/mnt/data/jobs_skills_normalised.csv"),
    ]:
        if p.exists(): JOBS_CANON = str(p); break

if not ROLES_ONTO:
    for p in [Path("/mnt/data/IT_Job_Roles_Skills.csv"), Path("data/IT_Job_Roles_Skills.csv")]:
        if p.exists(): ROLES_ONTO = str(p); break

print(f"[info] [try] Phase-3 resumes: {RESUMES_CANON}")
print(f"[info] [try] Phase-3 jobs   : {JOBS_CANON if JOBS_CANON else '(optional)'}")
print(f"[info] [try] Ontology roles : {ROLES_ONTO}")

# 2) If resumes missing, discover RAW and build a minimal canonical file
RAW_RESUMES = None
if RESUMES_CANON is None:
    hint = os.getenv("RESUME_RAW_HINT", "").strip()
    if hint and Path(hint).exists(): RAW_RESUMES = Path(hint)
    if RAW_RESUMES is None:
        for cand in [Path("/mnt/data/resume_dataset_1200.csv"), Path("resume_dataset_1200.csv")]:
            if cand.exists(): RAW_RESUMES = cand; break
    if RAW_RESUMES is None:
        candidates = list_matches(["*resume*1200*.csv","*resum*1200*.csv","*resume_dataset*.csv","*cv*1200*.csv"])
        valid = [p for p in candidates if looks_like_resume_csv(p)]
        valid.sort(key=lambda p:(0 if str(p).lower().startswith(str(Path('/mnt/data')).lower()) else 1, len(str(p))))
        if valid: RAW_RESUMES = valid[0]

    if RAW_RESUMES is None:
        some_csvs = list_matches(["*.csv"], limit=60)
        sample = "\n".join(f" - {p}" for p in some_csvs[:25])
        raise FileNotFoundError("No Phase-3 resumes and no raw résumé CSV found.\n"
                                "Set RESUME_RAW_HINT or place the raw file under /mnt/data.\n"
                                f"Visible CSVs:\n{sample}")

    print(f"[warn] [fallback] Building minimal Phase-3 résumés from: {RAW_RESUMES}")
```

```python
df_raw = pd.read_csv(RAW_RESUMES, encoding="utf-8-sig", sep=None, engine="python")

def _maybe_json_list(x):
    if x is None or (isinstance(x,float) and math.isnan(x)): return []
    if isinstance(x, list): return [str(i).strip() for i in x if str(i).strip()]
    s = str(x).strip()
    if not s: return []
    try:
        v = json.loads(s);
        if isinstance(v, list): return [str(i).strip() for i in v if str(i).strip()]
    except Exception: pass
    return [p.strip(" '\"") for p in re.split(r"[;,]", s) if p.strip()]

def first_nonempty(row, cols):
    for c in cols:
        v = row.get(c)
        if isinstance(v, str) and v.strip(): return v.strip()
    return ""

title_cols   = [c for c in df_raw.columns if re.search(r"(title|designation|role)", c, re.I)]
summary_cols = [c for c in df_raw.columns if re.search(r"(summary|objective|about|profile|description)", c, re.I)]
skill_cols   = [c for c in df_raw.columns if re.search(r"skill", c, re.I)]

def combine_skills(row):
    skills=[];
    for c in skill_cols: skills += _maybe_json_list(row.get(c))
    seen,out=set(),[]
    for s in skills:
        low=s.lower()
        if low not in seen: seen.add(low); out.append(s)
    return out

tmp = pd.DataFrame({
    "title_any":   df_raw.apply(lambda r: first_nonempty(r, title_cols), axis=1) if title_cols else "",
    "summary_any": df_raw.apply(lambda r: first_nonempty(r, summary_cols), axis=1) if summary_cols else "",
    "skills_canon": df_raw.apply(combine_skills, axis=1) if skill_cols else [[] for _ in range(len(df_raw))]
})
tmp.insert(0, "resume_id", np.arange(1, len(tmp)+1))
Path("outputs/phase3").mkdir(parents=True, exist_ok=True)
RESUMES_CANON = "outputs/phase3/resumes_with_skills_canonical.csv"
tmp.to_csv(RESUMES_CANON, index=False, encoding="utf-8-sig")
```

```
    print(f"[warn] [fallback] Wrote minimal canonical resumes -> {RESUMES_CANON}")

# 3) If ontology missing, adopt closest and mirror-copy
if ROLES_ONTO is None or not Path(ROLES_ONTO).exists():
    nearby = list_matches(["*IT*Job*Roles*Skills*.csv","*roles*skills*.csv","*IT*Roles*.csv"])
    if not nearby:
        raise FileNotFoundError("Roles ontology CSV not found (e.g., /mnt/data/IT_Job_Roles_Skills.csv).")
    nearby.sort(key=lambda p:(0 if str(p).lower().startswith(str(Path('/mnt/data')).lower()) else 1, len(str(p))))
    chosen = nearby[0]
    print(f"[warn] [fallback] Adopting discovered ontology: {chosen}")
    ROLES_ONTO = mirror_adopt(chosen, "IT_Job_Roles_Skills.csv")

# 4) Normalise to absolute POSIX paths (prevents backslash issues)
RESUMES_CANON = Path(RESUMES_CANON).resolve().as_posix()
ROLES_ONTO    = Path(ROLES_ONTO).resolve().as_posix()
if JOBS_CANON: JOBS_CANON = Path(JOBS_CANON).resolve().as_posix()

print(f"[info] [input✓] resumes: {RESUMES_CANON}")
print(f"[info] [input✓] jobs:    {JOBS_CANON if JOBS_CANON else '(optional/not found)'}")
print(f"[info] [input✓] roles:   {ROLES_ONTO}")
```

```
[info] [try] Phase-3 resumes: None
[info] [try] Phase-3 jobs   : outputs\phase3\jobs_skills_normalised.csv
[info] [try] Ontology roles : None
[warn] [fallback] Building minimal Phase-3 résumés from: c:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\data\raw\resume_da
taset_1200.csv
[warn] [fallback] Wrote minimal canonical resumes -> outputs/phase3/resumes_with_skills_canonical.csv
[warn] [fallback] Adopting discovered ontology: c:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\data\raw\IT_Job_Roles_Skill
s.csv
[warn] [mirror] Copied to outputs\phase0\data_cache\IT_Job_Roles_Skills.csv
[info] [input✓] resumes: C:/Users/naikt/OneDrive/Desktop/tanmay_dissertation/outputs/phase3/resumes_with_skills_canonical.csv
[info] [input✓] jobs:    C:/Users/naikt/OneDrive/Desktop/tanmay_dissertation/outputs/phase3/jobs_skills_normalised.csv
[info] [input✓] roles:   C:/Users/naikt/OneDrive/Desktop/tanmay_dissertation/outputs/phase0/data_cache/IT_Job_Roles_Skills.csv
```

In [14]:

```
# ==========================================
# Cell 3 — Load CSVs safely & unify key fields
# ==========================================
import json, re, numpy as np, pandas as pd

def read_csv_safe(path):
    # tolerate BOM + infer delimiter; fallback if needed
```

```
    try:
        return pd.read_csv(path, encoding="utf-8-sig", sep=None, engine="python")
    except Exception:
        try:
            return pd.read_csv(path, encoding="utf-8", sep=None, engine="python")
        except Exception:
            return pd.read_csv(path, encoding="latin-1", sep=None, engine="python")

def _maybe_json_list(x):
    if x is None or (isinstance(x,float) and math.isnan(x)): return []
    if isinstance(x, list): return [str(i).strip() for i in x if str(i).strip()]
    s = str(x).strip()
    if not s: return []
    try:
        v = json.loads(s)
        if isinstance(v, list): return [str(i).strip() for i in v if str(i).strip()]
    except Exception: pass
    return [p.strip(" '\"") for p in re.split(r"[;,]", s) if p.strip()]

# --- read dataframes
res_df   = read_csv_safe(RESUMES_CANON)
roles_df = read_csv_safe(ROLES_ONTO)
jobs_df  = read_csv_safe(JOBS_CANON) if 'JOBS_CANON' in globals() and JOBS_CANON else None

# --- Résumés: ensure skills_canon / title_any / summary_any / resume_id
skill_cols = [c for c in res_df.columns if re.search(r"skill", c, re.I)]
if "skills_canon" not in res_df.columns:
    def combine_skills_row(r):
        skills=[];
        for c in skill_cols: skills += _maybe_json_list(r.get(c))
        seen,out=set(),[]
        for s in skills:
            low=s.lower()
            if low not in seen: seen.add(low); out.append(s)
        return out
    res_df["skills_canon"] = res_df.apply(combine_skills_row, axis=1)

title_cols   = [c for c in res_df.columns if re.search(r"(title|designation|role|target)", c, re.I)]
summary_cols = [c for c in res_df.columns if re.search(r"(summary|objective|about|profile|description)", c, re.I)]
def first_nonempty(row, cols):
    for c in cols:
```

```
        v = row.get(c)
        if isinstance(v, str) and v.strip(): return v.strip()
    return ""
if "title_any"   not in res_df.columns: res_df["title_any"]   = res_df.apply(lambda r: first_nonempty(r, title_cols),   axis=1
if "summary_any" not in res_df.columns: res_df["summary_any"] = res_df.apply(lambda r: first_nonempty(r, summary_cols), axis=1
if "resume_id"   not in res_df.columns: res_df["resume_id"]   = np.arange(1, len(res_df)+1)

# --- Roles: ensure role_id / role_title / role_desc / skills_ont
if "role_id" not in roles_df.columns:
    roles_df["role_id"] = np.arange(1, len(roles_df)+1)
name_candidates = [c for c in roles_df.columns if re.search(r"(title|role_name|name)", c, re.I)]
desc_candidates = [c for c in roles_df.columns if re.search(r"(desc|summary|about)", c, re.I)]
ont_skill_cols  = [c for c in roles_df.columns if re.search(r"skill", c, re.I)]
roles_df["role_title"] = roles_df[name_candidates[0]].astype(str) if name_candidates else roles_df["role_id"].astype(str)
roles_df["role_desc"]  = roles_df[desc_candidates[0]].astype(str) if desc_candidates else ""
roles_df["skills_ont"] = (
    roles_df.apply(lambda r: sum([_maybe_json_list(r.get(c)) for c in ont_skill_cols], []), axis=1)
    if ont_skill_cols else [[] for _ in range(len(roles_df))]
)

print(f"[info] Resumes loaded: {len(res_df)}; Roles loaded: {len(roles_df)}; Jobs loaded: {len(jobs_df) if jobs_df is not None
```

```
[info] Resumes loaded: 1200; Roles loaded: 493; Jobs loaded: 1167
```

In [15]:

```
# ========================================================
# Cell 4 — Render text for embedding (résumés & roles)
# ========================================================
def render_skill_sentence(skills: List[str], prefix: str="skills"):
    if not skills: return ""
    return f"{prefix}: " + ", ".join(skills[:128]) + "."

def render_resume_text(row) -> str:
    parts=[]
    if row.get("title_any"):   parts.append(f"title: {row['title_any']}.")
    if row.get("summary_any"): parts.append(f"summary: {row['summary_any']}.")
    parts.append(render_skill_sentence(row.get("skills_canon", []), "skills"))
    return " ".join([p for p in parts if p]).strip()

def render_role_text(row) -> str:
    parts=[]
    if row.get("role_title"): parts.append(f"role: {row['role_title']}.")
```

```
    if row.get("role_desc"):  parts.append(f"description: {row['role_desc']}.")
    if isinstance(row.get("skills_ont"), list) and row["skills_ont"]:
        parts.append(render_skill_sentence(row["skills_ont"], "core_skills"))
    return " ".join([p for p in parts if p]).strip()

res_df["text_embed"]   = res_df.apply(render_resume_text, axis=1)
roles_df["text_embed"] = roles_df.apply(render_role_text, axis=1)

log.info(f"Resume example: {res_df['text_embed'].iloc[0][:200]}...")
log.info(f"Role example:   {roles_df['text_embed'].iloc[0][:200]}...")
```

```
[INFO] Resume example: title: nan. summary: Seeking a challenging role as a Software Developer where I can apply my skills and
knowledge to contribute to organizational success and professional growth.. skills: [, ', N, o, ...
[INFO] Role example:   role: Admin Big Data. description: Responsible for managing and overseeing big data infrastructure, ensu
ring data integrity, security, and availability. Requires expertise in Hadoop, Spark, and data w...
[INFO] Role example:   role: Admin Big Data. description: Responsible for managing and overseeing big data infrastructure, ensu
ring data integrity, security, and availability. Requires expertise in Hadoop, Spark, and data w...
```

In [16]:

```
# ========================================================
# Cell 5 — Load embedding model (with graceful fallback)
# ========================================================
DEFAULT_EMB_MODEL = os.getenv("SBERT_MODEL", "sentence-transformers/all-MiniLM-L6-v2")
EMB_DIM, EMB_BACKEND = None, None

try:
    from sentence_transformers import SentenceTransformer
    import torch
    device = "cuda" if torch.cuda.is_available() else "cpu"
    model = SentenceTransformer(DEFAULT_EMB_MODEL, device=device)
    EMB_BACKEND = f"sentence-transformers:{DEFAULT_EMB_MODEL}"
    _probe = model.encode(["probe"], normalize_embeddings=False, convert_to_numpy=True)
    EMB_DIM = int(_probe.shape[1])
    log.info(f"Loaded {DEFAULT_EMB_MODEL} on {device} (dim={EMB_DIM})")

    def embed_texts(texts: List[str], batch_size: int = 256, normalize: bool = True) -> np.ndarray:
        return model.encode(
            texts,
            batch_size=batch_size,
            normalize_embeddings=normalize,
            convert_to_numpy=True,
            show_progress_bar=TQDM
```

```
        )
except Exception as e:
    log.warning(f"SentenceTransformer unavailable ({e}). Falling back to TF-IDF.")
    from sklearn.feature_extraction.text import TfidfVectorizer
    from sklearn.preprocessing import normalize as sknorm
    vectorizer = TfidfVectorizer(min_df=3, max_df=0.95, ngram_range=(1,2))
    EMB_BACKEND = "tfidf"
    EMB_DIM = None
    def embed_fit(texts: List[str]):
        X = vectorizer.fit_transform(texts)
        return X.astype(np.float32)
    def embed_transform(texts: List[str]):
        X = vectorizer.transform(texts).astype(np.float32)
        return sknorm(X, norm="l2", copy=False)
    def embed_texts(texts: List[str], batch_size: int = 0, normalize: bool = True):
        return embed_transform(texts)
```

```
[INFO] Load pretrained SentenceTransformer: sentence-transformers/all-MiniLM-L6-v2
Batches: 100%|██████████| 1/1 [00:00<00:00,  7.12it/s]
[INFO] Loaded sentence-transformers/all-MiniLM-L6-v2 on cpu (dim=384)
```

In [17]:

```
# ============================================================
# Cell 6 — Compute résumé embeddings (+ ids/text/skills saves)
# ============================================================
resume_ids   = res_df["resume_id"].astype(str).tolist()
resume_texts = res_df["text_embed"].astype(str).fillna("").tolist()

if EMB_BACKEND == "tfidf":
    # Fit on resumes + roles together to share vector space
    all_texts = resume_texts + roles_df["text_embed"].astype(str).fillna("").tolist()
    X_all = embed_fit(all_texts)
    from sklearn.preprocessing import normalize as sknorm
    X_res = sknorm(X_all[:len(resume_texts)], norm="l2", copy=False)
    resume_vecs = X_res
    EMB_DIM = X_res.shape[1]
else:
    resume_vecs = embed_texts(resume_texts, batch_size=256, normalize=True)

np.save(OUTDIR / "resume_embeddings.npy", resume_vecs if isinstance(resume_vecs, np.ndarray) else resume_vecs.toarray())
pd.DataFrame({"resume_id": resume_ids}).to_csv(OUTDIR / "resume_ids.csv", index=False)
pd.DataFrame({"resume_id": resume_ids, "text_embed": resume_texts}).to_csv(OUTDIR / "resume_text.csv", index=False)
```

```
pd.DataFrame({"resume_id": resume_ids, "skills_canon": res_df["skills_canon"].apply(lambda xs: json.dumps(xs, ensure_ascii=Fal

log.info(f"[saved] resume_embeddings.npy shape = {np.asarray(resume_vecs).shape}")
```

```
Batches: 100%|██████████| 5/5 [00:33<00:00,  6.66s/it]
[INFO] [saved] resume_embeddings.npy shape = (1200, 384)
```

In [18]:

```
# ======================================================
# Cell 7 — Compute role embeddings (+ ids/text/skills)
# ======================================================
role_ids   = roles_df["role_id"].astype(str).tolist()
role_texts = roles_df["text_embed"].astype(str).fillna("").tolist()

if EMB_BACKEND == "tfidf":
    from sklearn.preprocessing import normalize as sknorm
    role_vecs = embed_transform(role_texts)
else:
    role_vecs = embed_texts(role_texts, batch_size=256, normalize=True)

np.save(OUTDIR / "role_embeddings.npy", role_vecs if isinstance(role_vecs, np.ndarray) else role_vecs.toarray())
pd.DataFrame({"role_id": role_ids}).to_csv(OUTDIR / "role_ids.csv", index=False)
pd.DataFrame({"role_id": role_ids, "text_embed": role_texts}).to_csv(OUTDIR / "role_text.csv", index=False)
pd.DataFrame({
    "role_id": role_ids,
    "role_title": roles_df["role_title"],
    "role_desc":  roles_df["role_desc"],
    "skills_ont": roles_df["skills_ont"].apply(lambda xs: json.dumps(xs, ensure_ascii=False))
}).to_csv(OUTDIR / "role_skills_ontology.csv", index=False)

log.info(f"[saved] role_embeddings.npy shape = {np.asarray(role_vecs).shape}")
```

```
Batches: 100%|██████████| 2/2 [00:04<00:00,  2.16s/it]
[INFO] [saved] role_embeddings.npy shape = (493, 384)
```

In [19]:

```
# ================================================================
# Cell 8 — Optional: skills-only sentence embeddings (ablations)
# ================================================================
BUILD_SKILLS_ONLY = True
if BUILD_SKILLS_ONLY:
    res_sk_texts  = res_df["skills_canon"].apply(lambda xs: "skills_only: " + ", ".join(xs) + ".").tolist()
    role_sk_texts = roles_df["skills_ont"].apply(lambda xs: "core_skills_only: " + ", ".join(xs) + ".").tolist()
```

```
    if EMB_BACKEND == "tfidf":
        from sklearn.feature_extraction.text import TfidfVectorizer
        from sklearn.preprocessing import normalize as sknorm
        vec2 = TfidfVectorizer(min_df=2, max_df=0.98, ngram_range=(1,2))
        Xall = vec2.fit_transform(res_sk_texts + role_sk_texts)
        Xall = sknorm(Xall, norm="l2", copy=False)
        Xr = Xall[:len(res_sk_texts)]
        Xo = Xall[len(res_sk_texts):]
        np.save(OUTDIR / "resume_embeddings_skills_only.npy", Xr.toarray())
        np.save(OUTDIR / "role_embeddings_skills_only.npy",   Xo.toarray())
    else:
        R_sk = embed_texts(res_sk_texts, batch_size=256, normalize=True)
        O_sk = embed_texts(role_sk_texts, batch_size=256, normalize=True)
        np.save(OUTDIR / "resume_embeddings_skills_only.npy", R_sk)
        np.save(OUTDIR / "role_embeddings_skills_only.npy",   O_sk)
    log.info("[saved] *_embeddings_skills_only.npy")
```

```
Batches: 100%|██████████| 5/5 [00:27<00:00,  5.40s/it]
Batches: 100%|██████████| 2/2 [00:02<00:00,  1.33s/it]
[INFO] [saved] *_embeddings_skills_only.npy
```

In [20]:

```
# ============================================================
# Cell 9 — Sanity check: nearest roles for a sample
# ============================================================
from sklearn.metrics.pairwise import cosine_similarity

def _todense(X):
    return X if isinstance(X, np.ndarray) else X.toarray()

RES = _todense(np.load(OUTDIR / "resume_embeddings.npy"))
ROL = _todense(np.load(OUTDIR / "role_embeddings.npy"))

SAMPLE_N = min(10, RES.shape[0])
idx = np.random.choice(RES.shape[0], size=SAMPLE_N, replace=False)
sim = cosine_similarity(RES[idx], ROL)
topk = 5

rows=[]
for i, ridx in enumerate(idx):
    r_id    = res_df.iloc[ridx]["resume_id"]
    r_title = res_df.iloc[ridx]["title_any"]
```

```
    order = np.argsort(-sim[i])[:topk]
    for rank, j in enumerate(order, 1):
        rows.append({
            "resume_id": r_id,
            "resume_title": r_title,
            "rank": rank,
            "role_id": roles_df.iloc[j]["role_id"],
            "role_title": roles_df.iloc[j]["role_title"],
            "score": float(sim[i, j])
        })

sanity_df = pd.DataFrame(rows)
sanity_df.to_csv(OUTDIR / "sanity_nearest_roles_sample.csv", index=False)
log.info(f"[sanity] saved nearest roles for {SAMPLE_N} resumes -> {OUTDIR / 'sanity_nearest_roles_sample.csv'}")
sanity_df.head(20)
```

```
[INFO] [sanity] saved nearest roles for 10 resumes -> outputs\phase04\sanity_nearest_roles_sample.csv
```

Out[20]:

| | resume_id | resume_title | rank | role_id | role_title | score |
|---|---|---|---|---|---|---|
| **0** | 1179 | Customer Success Manager | 1 | 421 | IT SALES EXECUTIVE | 0.538864 |
| **1** | 1179 | Customer Success Manager | 2 | 422 | IT SALES DIRECTOR | 0.505520 |
| **2** | 1179 | Customer Success Manager | 3 | 432 | TECHNOLOGY OFFICER | 0.501052 |
| **3** | 1179 | Customer Success Manager | 4 | 374 | SEO MANAGER | 0.496683 |
| **4** | 1179 | Customer Success Manager | 5 | 325 | PRODUCT MANAGER | 0.492140 |
| **5** | 866 | NaN | 1 | 248 | DATA ANALYST | 0.522580 |
| **6** | 866 | NaN | 2 | 337 | BUSINESS INTELLIGENCE ANALYST | 0.484126 |
| **7** | 866 | NaN | 3 | 408 | DATABASE ANALYST | 0.470164 |
| **8** | 866 | NaN | 4 | 287 | INFORMATION SECURITY ANALYST | 0.448844 |
| **9** | 866 | NaN | 5 | 71 | DATA ANALYST | 0.434055 |
| **10** | 102 | Cloud Engineer | 1 | 57 | Cloud engineer | 0.729875 |
| **11** | 102 | Cloud Engineer | 2 | 58 | Cloud network engineer | 0.706671 |
| **12** | 102 | Cloud Engineer | 3 | 56 | Cloud automation engineer | 0.704810 |
| **13** | 102 | Cloud Engineer | 4 | 55 | CLOUD ARCHITECT | 0.696253 |
| **14** | 102 | Cloud Engineer | 5 | 381 | CLOUD COMPUTING SPECIALIST | 0.693596 |
| **15** | 440 | Cybersecurity Engineer | 1 | 387 | CYBERSECURITY ENGINEER | 0.653794 |
| **16** | 440 | Cybersecurity Engineer | 2 | 392 | SECURITY ENGINEER | 0.611495 |
| **17** | 440 | Cybersecurity Engineer | 3 | 339 | CYBERSECURITY ANALYST | 0.610055 |
| **18** | 440 | Cybersecurity Engineer | 4 | 388 | INFOSEC ANALYST/CYBERSECURITY ANALYST | 0.607718 |
| **19** | 440 | Cybersecurity Engineer | 5 | 287 | INFORMATION SECURITY ANALYST | 0.606171 |

In [21]:

```
# ============================================
# Cell 10 — Manifest for Phase-5 and later
```

```
# ==============================================
manifest = {
    "phase": 4,
    "seed": SEED,
    "embedding_backend": EMB_BACKEND,
    "embedding_model": os.getenv("SBERT_MODEL", "sentence-transformers/all-MiniLM-L6-v2") if EMB_BACKEND.startswith("sentence-
    "outputs": {
        "resume_embeddings": str((OUTDIR / "resume_embeddings.npy").resolve()),
        "role_embeddings":   str((OUTDIR / "role_embeddings.npy").resolve()),
        "resume_ids":        str((OUTDIR / "resume_ids.csv").resolve()),
        "role_ids":          str((OUTDIR / "role_ids.csv").resolve()),
        "resume_text":       str((OUTDIR / "resume_text.csv").resolve()),
        "role_text":         str((OUTDIR / "role_text.csv").resolve()),
        "resume_skills_canon":   str((OUTDIR / "resume_skills_canon.csv").resolve()),
        "role_skills_ontology":  str((OUTDIR / "role_skills_ontology.csv").resolve()),
        "sanity_nearest_roles_sample": str((OUTDIR / "sanity_nearest_roles_sample.csv").resolve()),
    }
}
with open(OUTDIR / "manifest.json", "w", encoding="utf-8") as f:
    json.dump(manifest, f, ensure_ascii=False, indent=2)
log.info(f"[saved] manifest.json")
manifest
```

```
[INFO] [saved] manifest.json
```

Out[21]:
```
{'phase': 4,
 'seed': 42,
 'embedding_backend': 'sentence-transformers:sentence-transformers/all-MiniLM-L6-v2',
 'embedding_model': 'sentence-transformers/all-MiniLM-L6-v2',
 'outputs': {'resume_embeddings': 'C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\resume_embeddi
ngs.npy',
  'role_embeddings': 'C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\role_embeddings.npy',
  'resume_ids': 'C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\resume_ids.csv',
  'role_ids': 'C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\role_ids.csv',
  'resume_text': 'C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\resume_text.csv',
  'role_text': 'C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\role_text.csv',
  'resume_skills_canon': 'C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\resume_skills_canon.cs
v',
  'role_skills_ontology': 'C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\role_skills_ontology.c
sv',
  'sanity_nearest_roles_sample': 'C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\sanity_nearest_
roles_sample.csv'}}
```

In [26]:
```python
from pathlib import Path
import json

OUTDIR = Path("outputs/phase04")
must_exist = [
    OUTDIR/"resume_embeddings.npy",
    OUTDIR/"role_embeddings.npy",
    OUTDIR/"resume_ids.csv",
    OUTDIR/"role_ids.csv",
    OUTDIR/"resume_text.csv",
    OUTDIR/"role_text.csv",
    OUTDIR/"resume_skills_canon.csv",
    OUTDIR/"role_skills_ontology.csv",
    OUTDIR/"manifest.json",
    OUTDIR/"sanity_nearest_roles_sample.csv",
]
missing = [str(p) for p in must_exist if not p.exists()]
print("[OK] All Phase-4 artefacts present." if not missing else "[MISSING] " + "\n".join(missing))

# Show manifest paths for quick eyeball
if (OUTDIR/"manifest.json").exists():
```

```
    print("\n[manifest]")
    print(json.dumps(json.loads((OUTDIR/"manifest.json").read_text()), indent=2)[:1500])
```

```
[OK] All Phase-4 artefacts present.

[manifest]
{
  "phase": 4,
  "seed": 42,
  "embedding_backend": "sentence-transformers:sentence-transformers/all-MiniLM-L6-v2",
  "embedding_model": "sentence-transformers/all-MiniLM-L6-v2",
  "outputs": {
    "resume_embeddings": "C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\resume_embeddings.npy",
    "role_embeddings": "C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\role_embeddings.npy",
    "resume_ids": "C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\resume_ids.csv",
    "role_ids": "C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\role_ids.csv",
    "resume_text": "C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\resume_text.csv",
    "role_text": "C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\role_text.csv",
    "resume_skills_canon": "C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\resume_skills_canon.cs
v",
    "role_skills_ontology": "C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\role_skills_ontology.c
sv",
    "sanity_nearest_roles_sample": "C:\\Users\\naikt\\OneDrive\\Desktop\\tanmay_dissertation\\outputs\\phase04\\sanity_nearest_
roles_sample.csv"
  }
}
```

In [28]:

```
import numpy as np, pandas as pd
from pathlib import Path

R_E = np.load("outputs/phase04/resume_embeddings.npy")
O_E = np.load("outputs/phase04/role_embeddings.npy")

rid = pd.read_csv("outputs/phase04/resume_ids.csv")
oid = pd.read_csv("outputs/phase04/role_ids.csv")

print(f"resume_embeddings shape: {R_E.shape}")
print(f"role_embeddings   shape: {O_E.shape}")
print(f"#resume_ids={len(rid)}, #role_ids={len(oid)}")

assert R_E.ndim == 2 and O_E.ndim == 2, "Embeddings must be 2-D arrays."
```

```
assert R_E.shape[1] == O_E.shape[1], "Resume and role embeddings must have the *same* dimensionality."
assert len(rid) == R_E.shape[0], "resume_ids length must match resume_embeddings rows."
assert len(oid) == O_E.shape[0], "role_ids length must match role_embeddings rows."
print("[OK] Embedding shapes and ID counts look consistent.")
```

```
resume_embeddings shape: (1200, 384)
role_embeddings   shape: (493, 384)
#resume_ids=1200, #role_ids=493
[OK] Embedding shapes and ID counts look consistent.
```

In [29]:

```
import numpy as np, pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

RES = np.load("outputs/phase04/resume_embeddings.npy")
ROL = np.load("outputs/phase04/role_embeddings.npy")
res_df = pd.read_csv("outputs/phase04/resume_text.csv")
rol_df = pd.read_csv("outputs/phase04/role_text.csv")

# sample a few résumés and compute top-5 roles
sample_n = min(10, RES.shape[0])
rng = np.random.default_rng(42)
idx = rng.choice(RES.shape[0], size=sample_n, replace=False)
sim = cosine_similarity(RES[idx], ROL)

rows=[]
for i, ridx in enumerate(idx):
    order = np.argsort(-sim[i])[:5]
    for rank, j in enumerate(order, 1):
        rows.append({
            "resume_row": int(ridx),
            "rank": rank,
            "score": float(sim[i, j]),
            "role_row": int(j),
            "resume_text_snip": res_df["text_embed"].iloc[ridx][:120],
            "role_text_snip":   rol_df["text_embed"].iloc[j][:120],
        })
sanity = pd.DataFrame(rows)
display(sanity.head(15))

# quick distribution check of top-1 similarities
top1 = sim.max(axis=1)
```

```
print(f"Top-1 cosine: mean={top1.mean():.3f}, median={np.median(top1):.3f}, min={top1.min():.3f}, max={top1.max():.3f}")
assert np.isfinite(sim).all(), "Non-finite cosine scores found."
print("[OK] Cosine sanity check computed.")
```

| | resume_row | rank | score | role_row | resume_text_snip | role_text_snip |
|---|---|---|---|---|---|---|
| **0** | 102 | 1 | 0.672113 | 337 | title: nan. summary: Targeting a Cloud Enginee... | role: CLOUD MIGRATION SPECIALIST. description:... |
| **1** | 102 | 2 | 0.606895 | 232 | title: nan. summary: Targeting a Cloud Enginee... | role: CLOUD ARCHITECT. description: Designs an... |
| **2** | 102 | 3 | 0.604793 | 411 | title: nan. summary: Targeting a Cloud Enginee... | role: CLOUD/SOFTWARE ARCHITECT. description: D... |
| **3** | 102 | 4 | 0.601118 | 55 | title: nan. summary: Targeting a Cloud Enginee... | role: Cloud automation engineer. description: ... |
| **4** | 102 | 5 | 0.593997 | 56 | title: nan. summary: Targeting a Cloud Enginee... | role: Cloud engineer. description: Builds and ... |
| **5** | 922 | 1 | 0.490788 | 247 | title: Financial Analyst. summary: Looking for... | role: DATA ANALYST. description: Analyzes data... |
| **6** | 922 | 2 | 0.470852 | 407 | title: Financial Analyst. summary: Looking for... | role: DATABASE ANALYST. description: Analyzes ... |
| **7** | 922 | 3 | 0.451949 | 13 | title: Financial Analyst. summary: Looking for... | role: Data Analysts. description: Analyzes dat... |
| **8** | 922 | 4 | 0.437642 | 379 | title: Financial Analyst. summary: Looking for... | role: POWER BI ANALYST. description: Analyzes ... |
| **9** | 922 | 5 | 0.429820 | 336 | title: Financial Analyst. summary: Looking for... | role: BUSINESS INTELLIGENCE ANALYST. descripti... |
| **10** | 106 | 1 | 0.660683 | 90 | title: Software Engineer. summary: Looking for... | role: Entry Level Software Engineer. descripti... |
| **11** | 106 | 2 | 0.653846 | 89 | title: Software Engineer. summary: Looking for... | role: Entry Level Software Developer. descript... |
| **12** | 106 | 3 | 0.646654 | 127 | title: Software Engineer. summary: Looking for... | role: Junior Software Engineer. description: A... |
| **13** | 106 | 4 | 0.638786 | 76 | title: Software Engineer. summary: Looking for... | role: Developer. description: Writes software ... |
| **14** | 106 | 5 | 0.634026 | 155 | title: Software Engineer. summary: Looking for... | role: Programmer. description: Writes code to ... |

```
Top-1 cosine: mean=0.550, median=0.578, min=0.294, max=0.672
[OK] Cosine sanity check computed.
```

In [30]:

```
import json, os
from pathlib import Path

m = json.loads(Path("outputs/phase04/manifest.json").read_text())
bad=[]
```

```python
for k, p in m.get("outputs", {}).items():
    if not Path(p).exists():
        bad.append((k, p))
print("[OK] Manifest paths exist." if not bad else "[BROKEN PATHS]\n" + "\n".join(f"{k}: {p}" for k,p in bad))
print("Embedding backend:", m.get("embedding_backend"))
```

```
[OK] Manifest paths exist.
Embedding backend: sentence-transformers:sentence-transformers/all-MiniLM-L6-v2
```